Department of Artificial Intelligence (AI) & Data Science

Name: Aaryan Nandkishor Raorane	**Roll no: S004**
Sap ID: 60018250070	**Topic: Data Structure**
Branch: Artificial Intelligence & Data Science	

Experiment No. 1

Aim: To implement a Stack using an array to simulate browser history and perform Push, Pop, Peek, and Display operations.

Theory:

Definition:

A Stack is a linear data structure that stores elements sequentially and allows insertion and removal only from one end called the top. The order of insertion and removal follows the Last in First out (LIFO) rule. The last element pushed onto the stack is always the first one to be removed. A stack works like a pile of trays in a cafeteria where trays can only be placed or picked up from the top.

Operations:

a) **Push:** It is used to insert a new element at the top of the stack.

Pseudocode:

```
BEGIN PUSH(url)
  IF tos == MAX - 1 THEN
    PRINT "Stack Overflow - Browser history is full"
  ELSE
    tos = tos + 1
    stack[tos] = url
```

```
        PRINT "Visited: url"
    END IF
END PUSH
```

b) **Pop:** It is used to remove the element from the top of the stack.

Pseudocode:

```
BEGIN POP()
    IF tos == -1 THEN
        PRINT "Stack Underflow - No history available"
    ELSE
        PRINT "Going back to: stack[tos]"
        tos = tos – 1
    END IF
END POP
```

c) **Peek:** It is used to read the value of the top element without removing it from the stack.

Pseudocode:

```
BEGIN PEEK()
    IF tos == -1 THEN
        PRINT "Stack is empty"
    ELSE
        PRINT "Current page: stack[tos]"
    END IF
END PEEK
```

d) **Display:** It is used to display all elements in the stack from top to bottom.

Pseudocode:

```
BEGIN DISPLAY()
   IF tos == -1 THEN
      PRINT "No browsing history"
   ELSE
      PRINT "Browser History (most recent first):"
      SET i = tos
      WHILE i >= 0 DO
         PRINT stack[i]
         i = i - 1
      END WHILE
   END IF
END DISPLAY
```

Code:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#define MAX 10

char stack[MAX][100];
int tos = -1;

void push(char url[])
{
   if(tos == (MAX-1))
   {
      printf("Stack Overflow - Browser history is full\n");
   }
```

```
    else
    {
        tos++;
        strcpy(stack[tos], url);
        printf("Visited: %s\n", url);
    }
}

void pop()
{
    if(tos == -1)
    {
        printf("Stack Underflow - No history available\n");
    }
    else
    {
        printf("Going back to: %s\n", stack[tos]);
        tos--;
    }
}

void peek()
{
    if(tos == -1)
    {
        printf("Stack is empty\n");
    }
    else
    {
        printf("Current page: %s\n", stack[tos]);
    }
}

void display()
{
    int i;
    if(tos == -1)
```

```
    {
        printf("No browsing history\n");
    }
    else
    {
        printf("Browser History (most recent first):\n");
        for(i = tos; i >= 0; i--)
        {
            printf("%s\n", stack[i]);
        }
    }
}

void main()
{
    int choice;
    char url[100];
    clrscr();
    printf("Browser History Simulation using Stack\n");
    do
    {
        printf("\n1. Visit Page (Push)");
        printf("\n2. Go Back (Pop)");
        printf("\n3. Current Page (Peek)");
        printf("\n4. Show History (Display)");
        printf("\n5. Exit");
        printf("\nEnter choice: ");
        scanf("%d", &choice);
        switch(choice)
        {
            case 1:
                printf("Enter URL: ");
                scanf("%s", url);
                push(url);
                break;
            case 2:
                pop();
```

```
            break;
        case 3:
            peek();
            break;
        case 4:
            display();
            break;
        case 5:
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice\n");
      }
   } while(choice != 5);
   getch();
}
```

Output:

```
Browser History Simulation using Stack

1. Visit Page (Push)
2. Go Back (Pop)
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: 1
Enter URL: https://example.com/
Visited: https://example.com/

1. Visit Page (Push)
2. Go Back (Pop)
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: 1
Enter URL: https://www.wikipedia.org/
```

```
Visited: https://www.wikipedia.org/

1. Visit Page (Push)
2. Go Back (Pop)
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: https://www.github.com/


Enter URL: Visited: https://www.github.com/

1. Visit Page (Push)
2. Go Back (Pop)
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: 1
Enter URL: https://www.stackoverflow.com/
Visited: https://www.stackoverflow.com/

1. Visit Page (Push)
2. Go Back (Pop)
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: 4
Browser History (most recent first):
https://www.stackoverflow.com/
https://www.github.com/
https://www.wikipedia.org/
https://example.com/

1. Visit Page (Push)
2. Go Back (Pop)
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: 3
Current page: https://www.stackoverflow.com/
```

```
1. Visit Page (Push)
2. Go Back (Pop)
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: 2
Going back to: https://www.stackoverflow.com/

1. Visit Page (Push)
2. Go Back (Pop)
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: 2
Going back to: https://www.github.com/

1. Visit Page (Push)
2. Go Back (Pop)
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: 2
Going back to: https://www.wikipedia.org/

1. Visit Page (Push)
2. Go Back (Pop)
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: 2
Going back to: https://example.com/

1. Visit Page (Push)
2. Go Back (Pop)
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: 2
Stack Underflow - No history available

1. Visit Page (Push)
2. Go Back (Pop)
```

```
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: 4
No browsing history

1. Visit Page (Push)
2. Go Back (Pop)
3. Current Page (Peek)
4. Show History (Display)
5. Exit
Enter choice: 5
Exiting...


=== Code Exited With Errors ===
```

Applications:

- Used in web browsers to store visited pages so the Back button works correctly.
- Used in text editors to support undo and redo operations.
- Used in function calls through the call stack to manage program execution.
- Used during expression evaluation and conversion such as infix to postfix.

Conclusion:

This program successfully implemented a Stack using an array to simulate browser history. The Push operation added web pages to the history, Pop removed the most recently visited page when Back was clicked, Peek checked the current top page, and Display showed the complete browsing history. The LIFO behaviour of the stack matched exactly with how a browser Back button is expected to work, confirming that a stack is the right choice for this task.